



南開大學
Nankai University

计算机学院

并行程序设计实验报告

GPU 编程实验

AMD AUP HIP 学习与 ANN 近似最近邻搜索 GPU 加速

任务一：HIP Lab1–Lab6 云平台学习

任务二：CUDA batch ANN 搜索与 Top- k 合并

姓名：柯云超

学号：2413575

专业：计算机科学与技术

GitHub 仓库：<https://github.com/kyc001/parallel>

2026 年 6 月 6 日

目录

摘要	3
1 实验说明与任务划分	3
1.1 两个独立任务	3
2 HIP 云平台学习	4
2.1 任务说明与课程内容概览	4
2.2 Notebook 已填答案表	4
2.3 共同优化经验	11
2.4 可迁移到 GPU 编程任务的经验	11
3 ANN GPU 实验背景与统一设置	12
3.1 问题背景与实验目标	12
3.2 实验平台、数据集与测量口径	12
3.3 总体设计原则	13
4 ANN GPU 加速问题建模	14
4.1 距离定义与评价指标	14
4.2 batch 矩阵乘法形式	14
4.3 Flat 搜索与 IVF 搜索	15
5 CUDA 程序设计与实现	15
5.1 程序结构	15
5.2 数据流与内存布局	16
5.3 GEMM score kernel	16
5.4 设备端 Top-k 合并	17
5.5 IVF batch 搜索	18
6 正确性与性能评估方法	18
6.1 编译与运行命令	18
6.2 输出字段与评价含义	19
6.3 本地工具链与重复测量结果	19
6.4 与 CPU 基线的加速比	21
7 实验分析	23
7.1 GEMM 路径分析	23
7.2 Top-k 路径分析	23
7.3 IVF 路径分析	23

7.4 数据传输与设备端计算	23
7.5 静态 profile 与汇编分析	24
8 总结	25
参考文献	26
A 学习通过截图	27
B AI 使用报告	30

摘要

本次 GPU 编程作业按两个独立任务组织。任务一是 AMD AUP HIP 云平台学习：在在线平台完成 HIP 课程 Lab1–Lab6 的 notebook 填空与运行验证，内容覆盖向量加法、矩阵转置、直方图、归约、蒙特卡罗积分和 K-Means。在线学习的通过截图放在附录中，正文保留代码补全和分析填空答案，并总结各实验对应的 GPU 编程模式：合并访存、共享内存 tile、两级归约、减少全局原子操作、grid-stride 循环、多 kernel 编排以及 warp/wave 级归约。

任务二是 ANN 近似最近邻搜索的 GPU 实现。根据作业要求，单个 query 难以充分利用 GPU，因此本实验采用 batch 查询，将内积距离计算转化为矩阵乘法：

$$S = BQ^T, \quad B \in \mathbb{R}^{N \times D}, \quad Q^T \in \mathbb{R}^{D \times M}.$$

其中 S 的每一列对应一个 query 与所有 base 向量的内积得分，再在该列中取 Top- k 即得到最近邻。本次实现保留前几次 ANN 实验的 header-only 风格，`main.cc` 仍只承担数据读取、搜索调用、Recall@10 统计和延迟输出；`gpu_ann_search.cuh` 作为聚合头，具体 CUDA 实现按职责拆分到 `gpu/` 目录。

程序实现了多条 GPU 搜索与对照路径。Flat 路径包含手写 16×16 tiled GEMM、cuBLAS SGEMM 基线、线程 0 合并 Top- k 基线以及共享内存树形合并 Top- k 优化版；IVF 路径包含 per-query probe list 基线和按 probe 簇分桶后的 grouped-IVF GEMM 扫描版本。3 次重复测量中，`cublas_tree` 全量扫描达到 Recall@10 = 0.99995、在线延迟 83.24 $\mu\text{s}/\text{query}$ ；`ivf_grouped 2000 16 4` 达到 Recall@10 = 0.96280、在线延迟 145.32 $\mu\text{s}/\text{query}$ ；`ivf_grouped 2000 16 8` 达到 Recall@10 = 0.99345、在线延迟 189.90 $\mu\text{s}/\text{query}$ 。报告围绕问题建模、CUDA 数据流、关键 kernel 设计、评价指标、重复实验、加速比与本地工具链验证展开。

1 实验说明与任务划分

1.1 两个独立任务

本次 GPU 编程实验由两个相对独立的任务组成。第一项任务是 AMD AUP 云端 HIP 学习：在在线 notebook 中完成 Lab1–Lab6 的代码补全、分析填空和运行验证，并在报告中整理主要填空答案、附上学习通过截图。第二项任务是自选 GPU 编程实现：本次实验选取前几次 ANN 实验延续使用的近似最近邻搜索任务，将原先主要运行在 CPU 上的距离计算和 Top- k 后处理迁移到 GPU。

全文按照上述两项任务组织：第 2 节记录 HIP 云平台学习内容；第 3 节起讨论 ANN GPU 实现，并分别说明问题背景、统一实验设置、程序设计、正确性验证和性能分析。两项任务对应不同验收材料：HIP 任务包括 notebook 填空整理和学习通过截图，ANN 任务包括 CUDA 代码设计、运行结果、Recall@10 和延迟分析。二者构成学习经验向 GPU ANN 实现的迁移关系，而非同一实验的连续步骤。

按照作业说明，HIP 云学习属于基础要求，ANN GPU 实现属于进阶选题；“研究报告篇幅不超过 12 页”按进阶选题主体理解。报告将 HIP 云学习材料与 ANN 研究报告主体分别组织：HIP notebook 填空与学习通过截图作为基础要求材料保留，ANN 部分采用独立研究报告结构控制篇幅和叙述重点。

2 HIP 云平台学习

2.1 任务说明与课程内容概览

HIP 云学习任务要求在 AMD AUP 云端平台完成 HIP 课程的六个 notebook。根据作业说明，报告重点记录 notebook 中的代码填空和分析填空；GPU 型号、设备名称等基础硬件信息不作为填空答案展开。据此，本节首先概括六个 Lab 的主题，再逐项整理主要填空答案。

AMD AUP HIP 课程从一维数据并行逐步过渡到多 kernel 协作应用。六个 Lab 的主题不同，但背后的 GPU 编程思路具有连续性：先建立线程索引与 grid/block 层次，再学习如何控制全局内存访问、如何借助共享内存复用数据、如何减少原子操作，以及如何把迭代型算法拆成多个 kernel。表 1 总结了每个实验的主要瓶颈和学习到的优化方法。

实验	主题	主要瓶颈	学习到的优化方法
Lab1	Vector Addition	算术强度低，性能接近显存带宽上限。	每个线程处理一个元素；连续线程访问连续地址形成合并访存；用 grid-stride loop 支持任意长度。
Lab2	Matrix Transpose	朴素转置会出现读或写不合并。	用共享内存 tile 中转，把全局访存改造成合并读和合并写；在 tile 维度加 padding 避免 bank conflict。
Lab3	Histogram	多线程同时更新同一 bin，导致全局原子争用。	先在 block 内构建局部直方图，再少量合并到全局；使用 occupancy API 辅助选择 block size。
Lab4	Reduction	归约需要同步，且最终合并常伴随原子操作。	共享内存树形归约把活跃线程逐步减半；进一步可用 warp shuffle 减少同步和共享内存访问。
Lab5	Monte Carlo	每个采样点独立，但最终求和容易被原子操作拖慢。	采样计算采用 embarrassingly parallel，求和使用两级归约；大规模浮点累加需要注意精度损失。
Lab6	K-Means	分配阶段反复读取质心，更新阶段存在原子累加。	用多 kernel 拆分分配、更新和收敛判断；质心适合放入共享内存，更新适合 block 内聚合后再写全局。

表 1 AMD AUP HIP Lab1–Lab6 学习内容概览

2.2 Notebook 已填答案表

本节以表格形式汇总 Lab1–Lab6 notebook 中除 GPU 型号、设备名称等基础硬件信息外的主要答案。索引、配置、复杂度和计算量类问题列出具体的数值；运行时间类结果受云端设备状态影响，表格保留 notebook 原有结果结构，并明确对应读取的输出字段或派生公式。

Lab1 Vector Addition

一维全局线程索引为

$$\text{idx} = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}.$$

相关索引与 launch 配置结果列于表 2。

blockIdx.x	blockDim.x	threadIdx.x	Global Index
0	256	100	$0 \times 256 + 100 = 100$
2	128	50	$2 \times 128 + 50 = 306$
5	64	0	$5 \times 64 + 0 = 320$

Block Size	Grid Size Formula	Grid Size	Total Threads	Idle Threads
64	$\lceil 1000/64 \rceil$	16	1024	24
128	$\lceil 1000/128 \rceil$	8	1024	24
256	$\lceil 1000/256 \rceil$	4	1024	24
512	$\lceil 1000/512 \rceil$	2	1024	24

表 2 Lab1 一维索引与 $N = 1000$ launch 配置完成表

Vector Add kernel 的补全代码如下。设备满载驻留线程数按下式估算：

$$\text{total resident threads} = \text{CU 数} \times \text{每 CU active wavefront 数} \times \text{wavefront size.}$$

例如 wave32、每 CU 64 个 active wavefront 时，16 CU 为 32768 线程，20 CU 为 40960 线程。

```

1 int idx = blockIdx.x * blockDim.x + threadIdx.x;
2 if (idx < N) {
3     C[idx] = A[idx] + B[idx];
4 }

```

场景	Wavefronts/Block	Wasted Lanes	结论
wave32, block 100	$\lceil 100/32 \rceil = 4$	$4 \times 32 - 100 = 28$	非 32 倍数 block 会浪费 lane。
wave64, block 100	$\lceil 100/64 \rceil = 2$	$2 \times 64 - 100 = 28$	在 wave64 下 100 仍不是完全对齐。
分支发散	-	-	同一 wavefront 内不同路由执行掩码分批执行，未激活 lane 暂停，吞吐下降。

Block Size	Wavefronts/Block	Blocks that fit	Active Wavefronts / Notes
32	1	64	64, 受 max blocks/CU 限制
64	2	32	64, 达到 wavefront 上限
128	4	16	64, 达到 wavefront 上限
256	8	8	64, 达到 wavefront 上限
512	16	4	64, 达到 wavefront 上限
1024	32	2	64, 达到 wavefront 上限

表 3 Lab1 wavefront 对齐与 occupancy 完成表

结果项	完成后的表格单元	来源或计算方式	分析结论
最低 mean execution time	取多次运行表中 Mean (ms) 最小的 block size	<code>vector_add_lab</code> 10 次运行均值	向量加法数据传输量固定，低时间通常对应高带宽。
最高 memory bandwidth	取带宽柱状图中最大的 block size	$12N/\text{Mean time}$	若误差条明显重叠，block size 间差异不宜判为显著。
Sub-wavefront 对照	block 8、16、32、256 的 Mean (ms)	云端 sub-wavefront 实验表	8 和 16 小于 wave32，通常比 wave-aligned block 更容易浪费 lane；在 MI100 wave64 下，8、16、32 均为 sub-wavefront。
Challenge stride	<code>blockDim.x * blockDim.x</code>	总线程数	grid-stride loop 允许固定数量线程覆盖任意大的 N 。

表 4 Lab1 性能记录项与 grid-stride 答案表

Lab2 Matrix Transpose

二维全局线程索引为

$$\begin{aligned} \text{idx} &= \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}, \\ \text{idy} &= \text{blockIdx.y} \times \text{blockDim.y} + \text{threadIdx.y}. \end{aligned}$$

表 5 给出索引题的完成结果。

blockIdx	threadIdx	Global (idx, idy), blockDim=(32,32)
(0,0)	(5,10)	$(0 \times 32 + 5, 0 \times 32 + 10) = (5, 10)$
(1,0)	(0,0)	$(1 \times 32 + 0, 0 \times 32 + 0) = (32, 0)$
(2,3)	(15,20)	$(2 \times 32 + 15, 3 \times 32 + 20) = (79, 116)$

Rows	Cols	Grid Size	Total Blocks
100	200	$(\lceil 200/32 \rceil, \lceil 100/32 \rceil) = (7, 4)$	$7 \times 4 = 28$

表 5 Lab2 二维索引与 block 数计算完成表

Thread (idx, idy)	Input Index	Output Index
(0,0)	$0 \times 6 + 0 = 0$	$0 \times 4 + 0 = 0$
(1,0)	$0 \times 6 + 1 = 1$	$1 \times 4 + 0 = 4$
(0,1)	$1 \times 6 + 0 = 6$	$0 \times 4 + 1 = 1$
(3,2)	$2 \times 6 + 3 = 15$	$3 \times 4 + 2 = 14$

表 6 Lab2 4×6 矩阵转置线性下标完成表

写下标使用 $\text{idx} * \text{rows} + \text{idy}$ ，因为输出矩阵尺寸为 $\text{cols} \times \text{rows}$ ，输出第 idx 行的行宽是原矩阵的 rows 。性能记录表采用 HIP event 打印的 `KERNEL_MS`，10 次运行后取 Mean (ms)，如表 7 所示。

Test	Dimensions	Elements	Time
1	16×16	256	云端 benchmark 输出的 Mean (ms)
2	128×32	4096	云端 benchmark 输出的 Mean (ms)
3	1×1024	1024	云端 benchmark 输出的 Mean (ms)
4	1001×2001	2003001	云端 benchmark 输出的 Mean (ms)

表 7 Lab2 矩阵转置性能记录表结构

大测试元素数约为 test 2 的 500 倍，但时间不会严格放大 500 倍，因为小测试主要受 kernel 启动开销支配，大测试才更能体现全局内存吞吐和非合并写开销。读取 `input[idy * cols + idx]` 时，相邻 `threadIdx.x` 访问连续地址，因此读是 coalesced；写入 `output[idx * rows + idy]` 时，相邻线程地址相差 `rows` 个 float，在 1024×1024 矩阵中即 4096 byte。若一次 memory transaction 为 128 byte，32 个线程非合并写通常需要约 32 次 transaction。tilted transpose 中 `tile[BLOCK_SIZE][BLOCK_SIZE + 1]` 的 +1 用于错开共享内存 bank 映射；BLOCK_SIZE=32 时每 block 使用 $32 \times (32 + 1) \times 4 = 4224$ byte 共享内存。

Lab3 Histogram

naive histogram 中每个 block 负责一个 bin 并完整扫描输入，所以每个输入元素会从全局内存读取 `num_bins` 次。对应分析结果见表 8。

问题	完成结果	说明
每个元素全局读取次数	<code>num_bins</code> 次	每个 bin 对全数组扫描一次。
示例读取次数	$N = 10\,000\,000$, <code>num_bins</code> = 256 时为 2.56×10^9	重复扫描导致 $O(N \times \text{num_bins})$ 工作量。
全局读流量估算	$2.56 \times 10^9 \times 4 = 10.24$ GB	按 int 4 byte 估算。
读流量下降比例	$(N \times \text{num_bins})/N = 256$	优化版每个元素只从全局内存读取一次。

表 8 Lab3 naive histogram 与内存流量分析完成表

问题	完成结果
shared-memory <code>atomicAdd</code> 为什么更快	LDS 延迟低于全局内存，且争用范围限制在 block 内。
merge step 中每个 bin 的全局 atomic 次数	等于 block 数；每个 block 对每个 bin 合并一次。
grid-stride loop 的作用	用有限线程覆盖任意长度输入，并让线程按总线程数为步长分担元素。
all-same 分布	性能下降，大量线程更新同一 bin，原子操作更容易串行化。
uniform 分布	contention 较低，原子更新分散到多个 bin。

表 9 Lab3 shared-memory 优化与原子争用分析完成表

Implementation	Time (testcase 2)	Time (testcase 4)	来源
Serial CPU	<code>exe_serial</code> 的 KERNEL_MS 均值	<code>exe_serial</code> 的 KERNEL_MS 均值	5 次运行
Naive GPU	<code>exe_bad</code> 的 KERNEL_MS 均 值	<code>exe_bad</code> 的 KERNEL_MS 均 值	5 次运行
Optimized GPU	<code>exe_optimized</code> 的 KERNEL_MS 均值	<code>exe_optimized</code> 的 KERNEL_MS 均值	5 次运行
Optimized vs Serial	$T_{\text{Serial}}/T_{\text{Optimized}}$		
Optimized vs Naive	$T_{\text{Naive}}/T_{\text{Optimized}}$		

表 10 Lab3 直方图性能与加速比记录表结构

Lab4 Reduction

问题	完成结果
block 内归约步数	1024 个元素、block size 256 时为 $\log_2 256 = 8$ 。
partial sums 数	block-level reduction 后剩余 $\lceil 1024/256 \rceil = 4$ 个 partial sums。
同步作用	<code>__syncthreads()</code> 保证每轮 stride 的共享内存写入完成后，下一轮线程再读取中间结果。
第一轮空闲比例	tree reduction 第一轮后有 1/2 的线程空闲。
warp shuffle 同步	同一 warp/wavefront 内线程锁步执行，shuffle 在寄存器间直接交换数据，因此不需要 <code>__syncthreads()</code> 。
多元素线程收益	每线程先连续累加多个输入，减少 block 数和最终原子次数，提高连续访存和算术密度。

表 11 Lab4 reduction 算法分析完成表

Block Size	Warps/Block	Occupancy	Execution Time
64	2	云端 <code>exe_benchmark</code> 输出值;理想估算可达 100%	云端输出的 ms
128	4	云端 <code>exe_benchmark</code> 输出值;理想估算可达 100%	云端输出的 ms
256	8	云端 <code>exe_benchmark</code> 输出值;理想估算可达 100%	云端输出的 ms
512	16	云端 <code>exe_benchmark</code> 输出值;理想估算可达 100%	云端输出的 ms
1024	32	云端 <code>exe_benchmark</code> 输出值;理想估算可达 100%	云端输出的 ms

表 12 Lab4 occupancy 与执行时间记录表结构

Block Size	Final Atomic 数量	性能含义
64	$\lceil 1\,000\,000/64 \rceil = 15625$	block 多, 最终 atomic 次数多。
1024	$\lceil 1\,000\,000/1024 \rceil = 977$	atomic 次数少, 但 block 内后半段空闲线程更多。

表 13 Lab4 final atomic 次数完成表

Higher occupancy 不一定总是更好。Reduction 后续步骤会让大量线程空闲, 且更多 block 会增加最终原子操作和调度开销, 因此需要同时看 occupancy、atomic 次数和实际 kernel time。

Lab5 Monte Carlo

问题	完成结果
每线程缩放原因	每线程直接生成积分贡献 $(b-a)y_i/n$, 全局 atomic 累加后得到积分结果; 优化版改为先求和再统一缩放以减少重复除法。
atomic 复杂度	总工作量 $O(N)$, 但对单个全局地址的 atomic 会形成串行瓶颈。
shared-memory 改法	每个 block 在共享内存中做局部树形归约, 最后由线程 0 做一次全局 <code>atomicAdd</code> , 或把 partial sums 交给第二个 kernel。
double 精度	约 15–16 位十进制有效数字。
float 大样本问题	$N = 100\,000\,000$ 时约 7 位有效数字可能不足, 容易发生低位贡献被舍入、误差累积和 atomic 顺序导致的结果波动。

表 14 Lab5 Monte Carlo 算法与精度分析完成表

手算项	完成结果
y 值和	$0.0625 + 0.25 + 0.5625 + 1.0 + 1.5625 + 2.25 + 3.0625 + 4.0 = 12.75$
期望积分结果	$(2/8) \times 12.75 = 3.1875$
与程序输出是否匹配	匹配, 截图中的程序输出为 3.1875。

表 15 Lab5 test 1 手算验证完成表

Test	Sample Count	Execution Time
4	100000	<code>time ./exe_montecarlo testcases/4.in</code> 输出
7	10000000	<code>time ./exe_montecarlo testcases/7.in</code> 输出
8	100000000	<code>time ./exe_montecarlo testcases/8.in</code> 输出

指标	完成结果
test 4 到 test 7 的样本数变化	$10000000/100000 = 100$ 倍
是否严格线性	通常接近线性但不严格线性, 原因包括固定启动开销、全局 atomic 争用、内存吞吐和调度开销。
理论有效计算效率	$1/(1 + 100) \approx 0.99\%$, 假设计算 1 cycle、atomic 100 cycles。
普通 block reduction atomic 数	block size 256、 $N = 1\,000\,000$ 时为 $\lceil 1\,000\,000/256 \rceil = 3907$
优化版 atomic 数	256 个 block 对应 256 次全局 atomic
grid-stride loop 步长	<code>blockDim.x * gridDim.x</code>

表 16 Lab5 扩展性与 atomic 数量完成表

$(b - a) / n_samples$ 只由 thread 0 在写出 block 结果时乘一次, 可以减少重复乘除和舍入次数, 并保证每个 block 只提交缩放后的局部积分贡献。

Lab6 K-Means

问题	完成结果
assignment 复杂度	$k = 100$ 时每线程复杂度为 $O(k)$, 即 100 次距离计算。
shared-memory atom-ics 原因	在 block 内聚合点对质心的 x 、 y 和 count 贡献, 再用少量 global atomics 合并, 降低全局原子争用。
空簇处理	保留上一轮 centroid, 避免除以 0 并保持算法继续迭代。
test 4 执行时间	<code>time ./exe_kmeans < testcases/4.in > /dev/null</code> 输出
每轮主导计算	assignment 阶段, 每个点与 $k = 50$ 个 centroid 计算距离, 复杂度为 $O(Nk)$ 。

表 17 Lab6 K-Means 基本问题完成表

指标	完成结果
Assignment 每点距离计算数	50
Assignment 总距离计算数	$50\,000 \times 50 = 2\,500\,000$
Update shared-memory atomic 数	$3 \times 50\,000 = 150\,000$
Update block 数	block size 256 时为 $\lceil 50\,000/256 \rceil = 196$
Update global atomic 数	$196 \times 50 \times 3 = 29\,400$

表 18 Lab6 $N = 50000, k = 50$ compute intensity 完成表

问题	完成结果
centroid 读取优化	将 centroid 数组按块加载到 shared memory，或使用适合广播读取的缓存/常量路径，减少全局内存重复访问。
$k = 1000$ 的 cache 影响	centroid 数据变大，cache pressure 和 shared-memory 占用增加，可能导致 cache miss 增多、occupancy 下降，并显著增加每线程工作量。
收敛阈值	$dx^2 + dy^2 < 1.0 \times 10^{-8}$
为什么检查收敛	质心稳定后可提前停止，避免总是运行满 <code>max_iterations</code> 。
双缓冲作用	在 <code>prev</code> 和 <code>new</code> centroid 之间交换缓冲区，保留上一轮质心用于比较，同时避免本轮更新覆盖旧值。

表 19 Lab6 内存访问与收敛检测完成表

2.3 共同优化经验

六个 Lab 体现了较完整的 GPU 优化分析流程。首先需要判断 kernel 是带宽受限、计算受限、同步受限还是原子争用受限。向量加法和矩阵转置主要关注访存模式；归约和直方图主要关注同步与原子操作；K-Means 同时包含访存复用、原子聚合和多 kernel 编排。其次，根据瓶颈选择局部优化手段：带宽型 kernel 优先改善合并访存并减少冗余读写；原子争用型 kernel 尽量将全局原子操作转化为 block 内归约；数据复用明显的 kernel 优先考虑共享内存。

此外，应避免在算法逻辑中硬编码 warp 或 wavefront 宽度。AMD GPU 上 wavefront 可能使用 wave32 或 wave64，CUDA/NVIDIA 常见 warp 为 32。为了保持代码可移植性，归约、shuffle 和分片扫描应优先使用 `warpSize` 或运行时设备属性。本次 ANN CUDA 代码虽然运行在 NVIDIA CUDA 环境，但 Top- k 合并和线程配置仍遵循这一原则：线程数经过设备共享内存限制裁剪，避免共享内存大小和固定线程数不匹配导致 kernel 启动失败。

2.4 可迁移到 GPU 编程任务的经验

HIP 云学习中涉及的若干优化方法可迁移至 ANN GPU 实现。矩阵转置实验表明，访存连续性会显著影响带宽利用率，因此 `ScoreGemmKernel` 将 score 以 `score[query][base]` 的形式存储，使每个 query 的候选得分连续，便于 Top- k kernel 顺序扫描。归约和直方图实验表明，全局写出和全局原子操作都应尽量减少，因此本次实现不将完整得分矩阵回传主机，而是在设备端先进行局部 Top-10，再在 block 内合并。K-Means 实验表明，迭代型或索引型算法往往需要多阶段 kernel 编排，因此 IVF 路径被拆成 centroid scoring、probe selection 和 candidate search 三个阶段。

HIP 云学习的通过截图见附录 A。以下章节讨论第二项任务，即 ANN 近似最近邻搜索的 GPU 实现。

3 ANN GPU 实验背景与统一设置

3.1 问题背景与实验目标

ANN (Approximate Nearest Neighbor) 近似最近邻搜索是前几次并行程序设计实验反复使用的任务。给定 base 向量集合

$$\mathcal{B} = \{x_0, x_1, \dots, x_{N-1}\}, \quad x_i \in \mathbb{R}^D,$$

以及 query 向量 q , 程序需要返回与 q 最接近的 top- k 个 base id。本课程框架采用内积距离, 距离越小表示内积越大。前几次实验已经分别从 SIMD、Pthread/OpenMP 和 MPI 角度优化了 ANN; 本次 ANN GPU 实现任务不再继续叠加 CPU 线程级优化, 而是关注 GPU 在批量查询场景中的吞吐优势。

GPU 加速 ANN 的关键问题是并行粒度。单个 query 虽然可以把 N 个 base 向量的距离计算并行展开, 但一次 kernel 启动、query 拷贝和 Top- k 后处理的固定开销较高; 将多个 query 组成 batch 后, base 与 query 的内积计算可以写成矩阵乘法, 从而把 query 间并行和 query 内并行同时暴露给 GPU。本实验的目标包括:

- 以 batch query 暴露足够并行度, 避免单 query kernel 启动开销过高;
- 将 base 与 query 的内积计算写成矩阵乘法形式, 并用 CUDA kernel 完成分块计算;
- 在 GPU 端直接完成每个 query 的 Top-10, 减少主机端排序和大矩阵回传;
- 保持原始 ANN 框架, 尽量不改变 `main.cc` 的数据读取、Recall@10 统计和输出结构;
- 实现 IVF batch 搜索路径, 以评估 coarse probe 与候选扫描的 GPU 并行化效果。

3.2 实验平台、数据集与测量口径

ANN GPU 实现任务使用 DEEP100K 数据集: base 向量数 $N = 100000$, 维度 $D = 96$, query 和 ground truth 使用课程提供的二进制文件。正式运行统一取前 `query_n=2000` 条 query, 返回 $k = 10$ 个候选。平台与工具链如表 20 所示。

项目	配置	用途
GPU 平台	NVIDIA GeForce RTX 4060 Laptop GPU, CUDA 12.4	本地 CUDA 实现、正确性验证和性能测试。
主机与编译环境	micromamba activate test, Windows, Visual Studio Build Tools 2022, nvcc, mingw32-make	编译 main.cc 与 header-only CUDA 模块。
数据集与规模	DEEP100K, $N = 100000$, $D = 96$, query_n=2000, $k = 10$	与前几次 ANN 实验保持相同数据和评价口径。
计时方式	CUDA event 统计设备端分段耗时, CPU wall time 统计含 setup 的端到端耗时	区分 base/index 准备、query copy、score/probe、Top- k /search 和 result copy。

表 20 ANN GPU 实验平台、数据集与测量口径

评价指标沿用前几次 ANN 实验: **average recall** 表示 Recall@10, **average latency (us)** 表示在线阶段平均单 query 延迟。GEMM 路径是全量扫描, 主要用于验证 GPU 距离计算与设备端 Top- k 合并的正确性和吞吐; IVF 路径是近似搜索, 主要观察 **nlist**、**nprobe**、per-query probe list 和 grouped-IVF 分桶策略控制下的召回率与候选扫描开销。

进阶实验的本地运行日志和静态分析输出统一保存在结果目录中, 主要来源如表 21 所示。正文中的结果表、柱状图、profile 表和汇编分析均来自这些文件。

类别	用途
重复运行日志与汇总	记录 gemm 、 gemm_tree 、 cublas 、 cublas_tree 、 ivf 和 ivf_grouped 的 3 次重复运行结果, 正文表格来自 gpu_variants_repeat_20260620_summary.csv 。
IVF 分组对照日志	记录 IVF 在 $nlist = 16$ 、 $nprobe=2/4/8$ 下 per-query probe list 与 grouped-IVF 的召回率、延迟和候选扫描耗时。
CPU 基线结果	取自前序 SIMD 实验 ann-SIMD/bench_results/windows_i9_13900h/baseline_flat.csv , 用于计算同数据集、同 $k = 10$ 口径下的相对加速比。
Kernel 资源占用	由 cuobjdump --dump-resource-usage 导出, 记录各 kernel 的寄存器、栈和静态共享内存用量。
PTX/SASS 输出	由 nvcc --ptx 与 cuobjdump --dump-sass 导出, 用于核对 GEMM、Top- k 和 grouped-IVF 相关 kernel 的访存、共享内存同步和 FMA 指令形态。

表 21 ANN GPU 进阶实验的数据与分析来源

3.3 总体设计原则

ANN GPU 实现遵循三个设计原则。第一, 突出 GPU 编程中的 kernel 组织和数据流。以往并行实验中常见的 VTune、汇编和多线程 profiling 对理解系统性能有帮助, 但本次 ANN GPU 选题的重点是 CUDA kernel 设计、数据传输开销和设备端计时, 因此性能分析以 CUDA event 分段计时为主。第二, 保持代码组织与前几次 ANN 实验一致。GPU 搜索函数以 header-only 形式提供给

`main.cc`, 使主程序框架稳定, 便于和原始版本对照。第三, 不把完整得分矩阵作为 CPU 后处理输入。对于 DEEP100K, 如果 $N = 100000, M = 2000$, 完整 float 得分矩阵包含 2×10^8 个元素, 单独占用约 762 MiB; 若再回传 CPU 排序, GPU 计算收益会被显存占用和 PCIe 传输开销抵消。因此本实验采用 query chunk 和设备端 Top- k 合并。

4 ANN GPU 加速问题建模

4.1 距离定义与评价指标

设 base data 为 $B \in \mathbb{R}^{N \times D}$, query batch 中有 M 个向量。课程框架采用内积距离:

$$d(x, q) = 1 - \sum_{j=0}^{D-1} x_j q_j.$$

因此距离越小表示内积越大。对第 i 个 query, 程序返回 $k = 10$ 个候选 id, 并与 ground truth 的前 10 个近邻比较, 得到

$$\text{Recall@10}_i = \frac{|R_i \cap G_i|}{10},$$

其中 R_i 为程序返回集合, G_i 为 ground truth 集合。最终输出所有 query 的平均 Recall@10。

实现中排序始终以 inner-product score 为准, 输出距离使用 $1 - \text{score}$ 只是为了与前序 ANN 实验的接口保持一致; 该变换对排序是严格单调的, 因此不会改变 Top- k 顺序。若数据向量已经做过归一化, 内积排序也可解释为 cosine 相似度排序; 若未归一化, 本文只主张与课程提供 ground truth 和内积距离口径的一致性, Recall@10 用该 ground truth 作最终判据。

4.2 batch 矩阵乘法形式

单个 query 的搜索可以看作把一个 D 维向量与 N 个 base 向量分别做内积。GPU 可以并行展开这 N 次内积, 但单 query 的 kernel 启动、query 拷贝和结果回传开销会占据较大比例。将多个 query 组成 batch 后, 距离计算可以写成矩阵乘法:

$$Q^\top \in \mathbb{R}^{D \times M}, \quad S = BQ^\top, \quad S \in \mathbb{R}^{N \times M}.$$

矩阵 S 的第 j 列就是所有 base 向量与第 j 个 query 的内积得分; 由于 $d = 1 - \text{score}$, 取每列得分最大的 k 个元素等价于取距离最小的 k 个点。

本实现没有一次性物化整个 $N \times M$ 矩阵, 而是使用 query chunk。每次处理 M_c 个 query, 在显存中分配 $M_c \times N$ 的 score tile; tile 计算完成后立即执行 Top-10, 再进入下一个 chunk。这样既保持了矩阵乘法建模, 又控制了显存占用。

图 1 给出了 batch GEMM 的线程映射。一个 16×16 CUDA block 负责 score tile 中的一个小矩形; `threadIdx.x` 展开 base 行, `threadIdx.y` 展开 query 行。沿 D 维的 96 个元素被拆成 6 个 16 维小段, 每轮把 base/query 小块装入共享内存, 随后每个线程用寄存器累加一个输出分数。这样的映射同时利用 query 间并行和 query 内 base 扫描并行, 避免单 query 只在一个方向暴露并行度。

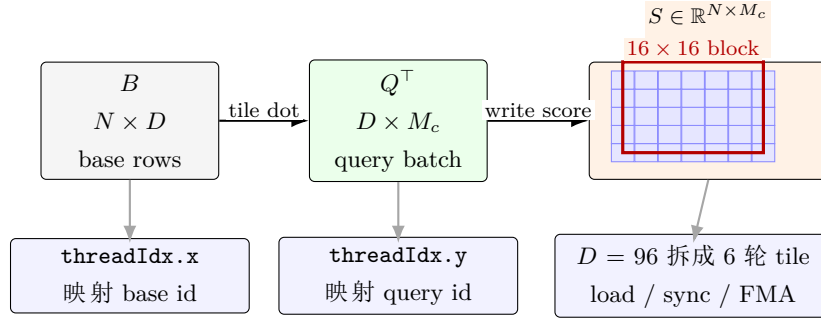


图 1 Batch GEMM 距离计算的二维并行映射

4.3 Flat 搜索与 IVF 搜索

gemm 模式对应 flat exhaustive search: 每个 query 与所有 base 向量计算内积，因此理论上在浮点误差之外不引入额外近似。它的优势是召回率稳定、kernel 结构规则，适合用 GEMM 风格 tile 提高吞吐；缺点是计算量固定为 $O(NMD)$ 。

ivf 模式对应 coarse-to-fine search: 先用 $nlist$ 个 centroid 把 base 划分到倒排表，再为每个 query 选择最近的 $nprobe$ 个簇，只在候选簇中搜索。它把在线计算量从全量 base 扫描降低为候选列表扫描，但会引入召回率和候选规模之间的折中。本实验实现 IVF 的目标不是替代成熟向量数据库索引，而是在课程框架内展示查询间并行、probe 选择和候选 Top- k 合并如何放入 GPU batch 流程。

5 CUDA 程序设计与实现

5.1 程序结构

本次实现保留原始 ANN 主程序形态，`main.cc` 仍是薄入口：读取 DEEP100K 数据、解析命令行参数、调用 GPU 搜索函数、统计 Recall@10 并输出延迟。GPU 相关代码以 header-only 方式组织，具体模块如表 22 所示。

文件	职责
gpu_ann_search.cuh	聚合头文件，向 main.cc 暴露 flat GEMM、cuBLAS、per-query IVF 和 grouped-IVF 搜索入口。
gpu/common.cuh	定义 GpuSearchStats、CUDA/cuBLAS 错误检查、CUDA event 计时、Top-10 结果堆转换和线程数裁剪。
gpu/gemm_kernel.cuh	实现 16×16 tiled GEMM kernel，作为手写矩阵乘法路径的 score tile 计算。
gpu/topk.cuh	实现设备端 Top-10 插入、线程局部候选维护、线程 0 合并基线、共享内存树形合并优化和带全局 id 的 grouped-IVF Top- k 。
gpu/gemm_search.cuh	实现 flat batch 搜索，包括手写 GEMM/cuBLAS 两种 score 路径、基础/树形两种 Top-10 路径和结果回传。
gpu/ivf_search.cuh	实现轻量 IVF 索引构建、centroid scoring、probe selection、per-query 候选扫描、按 probe 簇分桶的 grouped-IVF GEMM 扫描和最终 Top-10 合并。
Makefile	使用 nvcc 编译 main.cc，链接 cublas，并把 gpu/*.cuh 作为依赖，支持 make 和 make clean。

表 22 GPU ANN 实现的模块划分

5.2 数据流与内存布局

图 2 展示了程序的数据流。base 向量或 IVF 索引在搜索开始时拷贝到显存，随后在所有 query chunk 中复用。query 按 chunk 从主机拷贝到设备；设备端先计算 score/probe，再执行 Top-10 或候选扫描；CPU 只接收每个 query 的 10 个 id 与距离。

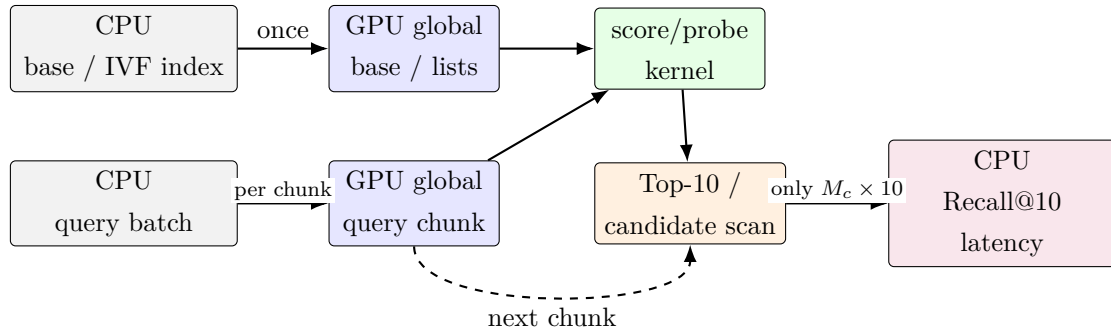


图 2 CUDA batch ANN 搜索的数据流与任务划分

在内存布局上，base 按 `base[base_id][dim]` 连续存储，query 按 `queries[query_id][dim]` 连续存储。score tile 则采用转置布局 `scores[query_id][base_id]`。这样做的原因是 Top- k kernel 每个 block 处理一个 query，若该 query 的所有 base 得分连续，线程分片扫描时全局内存访问更规则，输出的 Top-10 也可以直接写入 `out_dist[query][slot]` 和 `out_id[query][slot]`。

5.3 GEMM score kernel

ScoreGemmKernel 使用 16×16 tile。block 的 x 方向对应 base 行， y 方向对应 query 行；每个线程负责一个 `(base_id, query_id)` 输出元素。kernel 在 D 维上按 tile 遍历，将 base tile 和 query tile 载入共享内存，再在寄存器中累加内积。

```

1 dim3 block(16, 16);
2 dim3 grid(DivUp(base_n, 16), DivUp(chunk, 16));
3 ScoreGemmKernel<<<grid, block>>>(d_base, d_queries, d_scores,
4                                     static_cast<int>(base_n),
5                                     static_cast<int>(chunk),
6                                     static_cast<int>(dim));

```

该 kernel 对应的数学运算为

$$\text{score}_{q,b} = \sum_{t=0}^{D-1} B_{b,t} Q_{q,t}.$$

由于 DEEP100K 的维度固定且远小于 base 数量，主要并行度来自 base 行和 query 行两个方向。共享内存 tile 减少了同一 block 内对 base/query 元素的重复读取；score 采用 query-major 存储则服务于下一步 Top-k。

为评估矩阵乘法优化空间，flat 路径加入 cuBLAS SGEMM 作为对照。实现中利用 cublasSgemm 计算同一批 query 与 base 的内积，并保持输出仍为 scores[query][base] 的 query-major 视图，后续 Top-k kernel 不需要改变接口。该设计将手写 tiled kernel 与成熟 GEMM 库置于相同 Recall 和相同 Top-k 后处理口径下，便于定量比较矩阵乘法优化效果。

5.4 设备端 Top-k 合并

Top-k 是 ANN 搜索中容易被忽视的后处理瓶颈。如果将完整 score tile 拷回 CPU，再为每个 query 排序，会产生大量无意义的数据传输。本实现保留两个 Top-k 版本用于对照。基础版 ScoreTopKKernel 每个 block 处理一个 query，block 内线程以 stride 方式扫描该 query 的所有 base 得分。每个线程在寄存器中维护自己的局部 Top-10，扫描结束后把局部候选写入共享内存，由线程 0 归并为最终 Top-10 并按距离写出。优化版 ScoreTopKTreeKernel 仍先由各线程产生局部 Top-10，但 block 内候选合并改成共享内存树形归并，逐轮把参与线程数减半，减少末端单线程顺序比较。

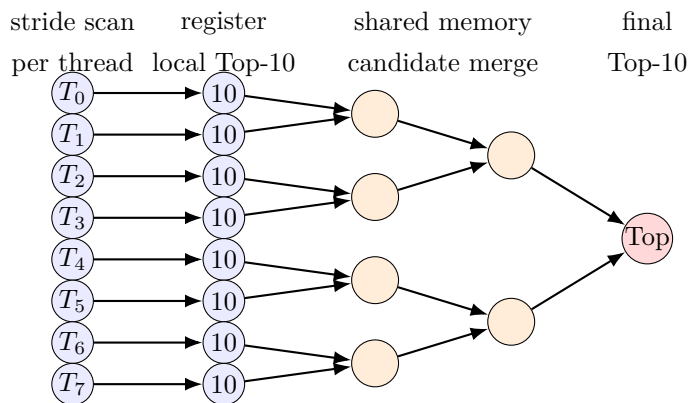
```

1 ScoreTopKKernel<<<static_cast<unsigned int>(chunk), threads,
2                                     shared_bytes>>>(d_scores, d_dist, d_ids,
3                                                     static_cast<int>(base_n));
4 ScoreTopKTreeKernel<<<static_cast<unsigned int>(chunk), threads,
5                                     shared_bytes>>>(d_scores, d_dist, d_ids,
6                                                     static_cast<int>(base_n));

```

该设计具有两个主要收益。第一，回传数据量从 $M_c \times N$ 个 float 降为 $M_c \times 10$ 个距离和 id。第二，Top-10 与 score tile 的生命周期绑定，GPU 完成 Top-10 后该 chunk 的 score buffer 即可复用，不需要长期保存全局得分矩阵。由于 $k = 10$ 很小，局部插入式维护 Top-k 比完整排序更合适，也避免了引入复杂的全局排序依赖。

图 3 用蝶形归并的形式概括优化版 Top-k 的信息流。实际实现中，每个线程先以 stride 方式扫描该 query 的若干 base 得分，并在寄存器中维护局部 Top-10；随后所有线程把局部候选写入共享内存。基础版由线程 0 对 blockDim.x * 10 个候选做一次顺序合并，优化版则让相邻线程的候选逐层合并，最后只留下线程 0 写出最终 Top-10。该实现仍不是完整排序，而是针对 $k = 10$ 的选择问题减少候选规模和串行尾部。

图 3 设备端 Top- k 的局部候选压缩与蝶形归并示意

5.5 IVF batch 搜索

`ivf` 和 `ivf_grouped` 模式共同实现了作业提示中提到的 IVF 查询并行。CPU 端先构建轻量 coarse quantizer: 选择 $nlist$ 个初始 centroid, 做少量 k-means 更新, 将 base 向量分配到最近 centroid, 并把每个 list 中的向量和原始 id 连续存放。per-query IVF 在线查询时, GPU 端执行三个阶段:

1. `ScoreGemmKernel` 计算 query chunk 到所有 centroid 的内积得分;
2. `SelectProbeKernel` 为每个 query 选择最近的 `nprobe` 个簇;
3. `IvfSearchKernel` 只扫描这些候选簇中的向量, 并在 block 内合并 Top-10。

IVF 的一个难点是 batch 内不同 query 的最近簇编号通常不同。本实验保留 per-query probe list 作为基线: 每个 query block 根据自己的 `nprobe` 列表扫描候选 list。该实现较为直接, 但不同 query 的候选集合和 list 顺序不一致, 候选扫描难以变成规则矩阵乘法。

为对照作业中提出的分组策略, `ivf_grouped` 模式在 probe selection 后按簇 id 对 batch 内 query 分桶。具体流程是: 先把每个 query 的 `nprobe` 结果拷回主机侧; 再为每个 list 收集需要探测该 list 的 query id, 并把这些 query 打包成连续小矩阵; 随后对同一个 list 的候选向量矩阵与该组 query 矩阵做一次 GEMM 式扫描; 最后使用带全局 id 的 `ScoreTopKWithIdsTreeKernel` 产生每个 query 在该 list 内的局部 Top-10, 并在主机端把同一 query 来自多个 probe list 的候选再合并成最终结果。该路径会增加分桶、重排和结果合并开销, 但能够将“一个簇里的向量写成矩阵”的思路落实到 batch 计算流程中。

从搜索质量看, flat 模式是全量扫描, 主要用于获得稳定的 Recall@10; IVF 模式通过 `nlist` 和 `nprobe` 控制候选规模, 体现近似搜索中速度与召回率的折中。推荐的初始 IVF 配置为 16 个 coarse list、每个 query 探测 4 个 list, 对应运行命令见第 6.1 节。

6 正确性与性能评估方法

6.1 编译与运行命令

运行前先进入本次实验使用的环境:

```
1 micromamba activate test
```

在 Linux 或已经配置 Visual Studio C++ host compiler 的 Windows CUDA 环境中，可以直接使用 Makefile。本机 Windows 环境已安装 Visual Studio Build Tools 到 C:\BuildTools，Makefile 会在 Windows 下自动调用 vcvars64.bat 加载 MSVC host compiler:

```
1 cd ann-gpu
2 mingw32-make
3 .\build\main.exe gemm 2000
```

也可以绕过 Makefile 直接调用 nvcc。Windows PowerShell 下需先加载 VS Build Tools 环境:

```
1 cmd /C "call C:\BuildTools\VC\Auxiliary\Build\vcvars64.bat && ^
2     nvcc -O3 -x cu main.cc -o build\main.exe -lcublas"
3 .\build\main.exe ivf 2000 16 4
```

命令行参数含义为: mode 可取 gemm、gemm_tree、cublas、cublas_tree、ivf 或 ivf_grouped，省略时默认为 gemm; query_n 表示参与搜索的 query 数，默认 2000; nlist 和 nprobe 仅用于 IVF 模式，默认分别为 16 和 4。典型对照命令如下:

```
1 .\build\main.exe cublas_tree 2000
2 .\build\main.exe ivf 2000 16 4
3 .\build\main.exe ivf_grouped 2000 16 4
```

6.2 输出字段与评价含义

程序输出示例如下:

```
1 gpu_gemm_batch_topk, device="...", query_n=2000, base_n=100000, dim=96
2 average recall: ...
3 average latency (us): ...
4 wall latency including setup (us): ...
5 base/index copy or build (ms): ...
6 query copy (ms): ...
7 score/probe compute (ms): ...
8 topk/search compute (ms): ...
9 result copy (ms): ...
```

其中 average recall 是正确性指标; average latency (us) 由 CUDA event 统计在线阶段平均延迟; wall latency including setup (us) 包含主机端调用和一次性准备成本。其余字段用于拆分性能来源: base/index 首次拷贝或构建、query chunk 拷贝、score/probe kernel、Top-10/候选扫描 kernel 以及结果回传。

6.3 本地工具链与重复测量结果

本地机器配置为 NVIDIA GeForce RTX 4060 Laptop GPU、CUDA 12.4 和 Visual Studio Build Tools 2022。由于普通 PowerShell 中 cl.exe 不默认进入 PATH，本实验在 Makefile 中为 Windows 增加了 C:\BuildTools\VC\Auxiliary\Build\vcvars64.bat 的自动加载逻辑。执行 mingw32-make clean 和 mingw32-make 后，验证结果表明 build/main.exe 可正常生成。

为降低单次运行偶然波动的影响，表 23 和表 24 均采用同一机器上连续 3 次运行的均值与标准差，原始日志保存在 `report/results/gpu_variants_repeat_20260620.txt`，汇总文件为 `report/results/gpu_variants_repeat_20260620_summary.csv`。其中 `online/us` 是程序输出的在线阶段平均单 query 延迟，`wall/us` 是包含 setup 的端到端平均单 query 延迟；延迟与分段耗时列写作均值 $\pm$ 标准差，Recall@10 三次运行完全一致，因此表中只列均值。

模式	次数	Recall@10	online/us	wall/us	score ms	Top-k ms
<code>gemm</code>	3	0.99990	237.63 $\pm$ 21.98	331.43 $\pm$ 19.30	240.08 $\pm$ 22.53	226.56 $\pm$ 17.40
<code>gemm_tree</code>	3	0.99995	165.05 $\pm$ 11.33	256.43 $\pm$ 31.26	229.94 $\pm$ 14.38	91.92 $\pm$ 4.21
<code>cublas</code>	3	0.99990	145.27 $\pm$ 1.10	233.59 $\pm$ 15.99	69.47 $\pm$ 1.34	214.49 $\pm$ 0.75
<code>cublas_tree</code>	3	0.99995	83.24 $\pm$ 2.12	162.95 $\pm$ 6.73	70.71 $\pm$ 1.85	89.44 $\pm$ 2.41

表 23 Flat GEMM 路径的重复测量结果。数据集为 DEEP100K，`query_n=2000`， $k = 10$ 。

表 23 对“更快的矩阵乘法”和“更快的 Top-k”两项优化方向进行了对照。手写 tiled GEMM 的 score 阶段为 240.08 ms，而 cuBLAS SGEMM 为 69.47 ms；在相同数据规模下，cuBLAS 把矩阵乘法阶段缩短到约 0.29 $\times$ 。Top-k 方面，原始 `ScoreTopKKernel` 的末端由线程 0 顺序合并 `blockDim.x * 10` 个候选；优化后的 `ScoreTopKTreeKernel` 改用共享内存树形合并，使 Top-k 阶段从 226.56 ms 降到 91.92 ms。两者叠加后，`cublas_tree` 的在线延迟降到 83.24 μ s/query，是本实验中 Recall 接近 1 的最快 flat 配置。

模式	nprobe	Recall@10	online/us	wall/us	score/probe ms	search/Top-k ms
<code>ivf</code>	2	0.87335	298.15 $\pm$ 35.46	494.82 $\pm$ 55.64	4.24 $\pm$ 2.92	582.22 $\pm$ 67.41
<code>ivf_grouped</code>	2	0.87340	94.01 $\pm$ 10.35	277.34 $\pm$ 7.24	53.75 $\pm$ 3.24	53.02 $\pm$ 3.09
<code>ivf</code>	4	0.96275	468.19 $\pm$ 34.50	688.25 $\pm$ 56.05	3.34 $\pm$ 0.34	922.68 $\pm$ 65.67
<code>ivf_grouped</code>	4	0.96280	145.32 $\pm$ 38.71	375.75 $\pm$ 61.74	87.72 $\pm$ 5.84	64.04 $\pm$ 4.53
<code>ivf</code>	8	0.99340	789.29 $\pm$ 89.28	987.22 $\pm$ 94.53	4.48 $\pm$ 1.76	1564.40 $\pm$ 172.34
<code>ivf_grouped</code>	8	0.99345	189.90 $\pm$ 36.41	419.52 $\pm$ 65.88	143.18 $\pm$ 5.68	89.95 $\pm$ 4.06

表 24 IVF per-query probe list 与 grouped-IVF 的对照实验。两条路径使用相同的 `nlist = 16` 和相同的 `nprobe`。

表 24 是针对作业中“让 batch 内最近簇编号重合度较高，避免计算资源浪费”的补充实验。per-query 路径中，每个 query block 直接扫描自己的 probe list，控制逻辑简单，但不同 query 的候选集合不一致，导致候选扫描 kernel 很难形成规则矩阵乘法形态。grouped-IVF 先在 GPU 上得到每个 query 的 `nprobe`，再按 probe 簇编号在主机侧分桶和重排；同一簇中的 query 被打包成一个小 batch，对该簇内向量矩阵执行一次 GEMM 式扫描，再用 `ScoreTopKWithIdsTreeKernel` 产生带全局 id 的局部 Top-10，最后在主机端按 query 合并各 probe 簇的结果。

实验结果显示，在相同 `nprobe` 下 grouped-IVF 的 Recall 与 per-query 基本一致，但在线延迟明显降低。以 `nprobe=4` 为例，per-query 的在线延迟为 468.19 μ s/query，grouped-IVF 为 145.32 μ s/query，约为前者的 0.31 $\times$ ；搜索/Top-k 阶段也从 922.68 ms 降到 64.04 ms。grouped-IVF 的代价是额外的 query 分桶、重排、结果拷贝和主机端合并，因此 `score/probe ms` 与 `result copy ms` 上升；但候选扫描变成更规则的批量矩阵乘法后，总体仍然更快。结果表明，分组策略是 IVF 在 GPU batch 场景中的重要优化点；后续仍可进一步将分桶与最终合并迁移至 GPU 端，以减少主机参与。

6.4 与 CPU 基线的加速比

为保持与前序 ANN 实验相同的评价口径，表 25 给出 GPU 实现相对 CPU Flat 基线的加速比。CPU 基线来自同一份 DEEP100K、query_n=2000、 $k=10$ 的 Windows i9-13900H SIMD 实验：串行 Flat 为 $4571.62 \mu\text{s}/\text{query}$ ，Flat-AVX2 为 $1756.11 \mu\text{s}/\text{query}$ ，二者 Recall@10 均为 0.99995。表中 online 加速比使用 GPU 输出的 average latency (us) 作为分母，表示 base/index 已准备好之后的在线查询吞吐；wall 加速比使用 wall latency including setup (us)，把一次性 base 拷贝或 IVF 构建也计入分母。

GPU 配置	Recall@10	online/us	wall/us	online vs 串行	online vs AVX2	wall vs 串行	wall vs AVX2
gemm	0.99990	237.63	331.43	19.24×	7.39×	13.79×	5.30×
cublas_tree	0.99995	83.24	162.95	54.92×	21.10×	28.05×	10.78×
ivf_grouped/4	0.96280	145.32	375.75	31.46×	12.08×	12.17×	4.67×
ivf_grouped/8	0.99345	189.90	419.52	24.07×	9.25×	10.90×	4.19×

表 25 GPU 实现相对 CPU Flat 基线的加速比。CPU 基线来自前序 SIMD 实验结果文件；GPU 数值使用 3 次重复测量均值。

加速比需结合 Recall 共同解释。cublas_tree 是最可比的一行：它和 CPU Flat 都是全量扫描，Recall 接近 1，在线阶段相对串行 Flat 达到 $54.92\times$ ，即使计入一次性 setup 仍有 $28.05\times$ 。ivf_grouped/8 在 Recall@10 = 0.99345 时在线延迟为 $189.90 \mu\text{s}/\text{query}$ ，相比串行 Flat 仍有 $24.07\times$ 加速；ivf_grouped/4 则体现更激进的速度/召回取舍。因此，评价 GPU 配置时需要同时给出 Recall、latency、setup 口径和相对基线。

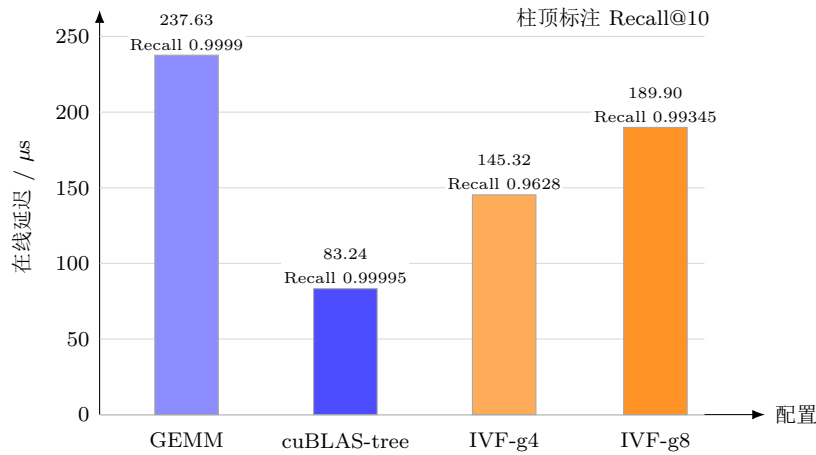


图 4 代表性 GPU 配置的在线延迟与 Recall@10，数据为 3 次重复测量均值

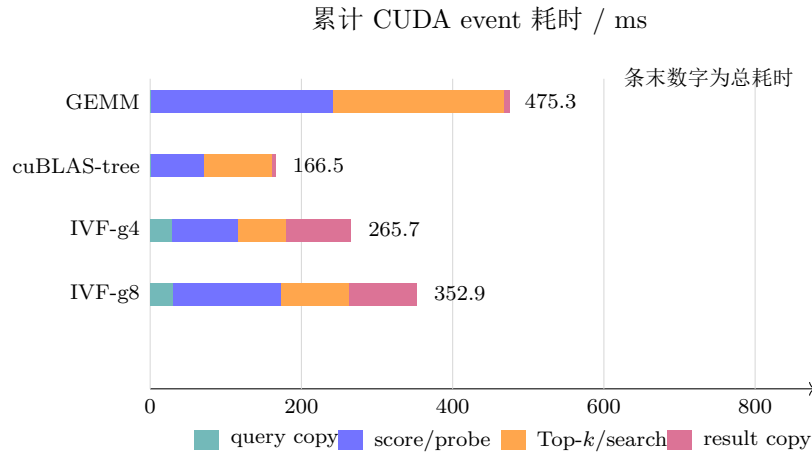


图 5 在线阶段 CUDA event 分段耗时，比例尺为 1 单位对应 100 ms

表 26 进一步根据重复运行日志推导各阶段的有效吞吐。GEMM score 阶段的工作量按 $2NMD$ 次浮点操作估算；Top- k 阶段按扫描得分个数估算；IVF 搜索阶段用于比较 per-query 与 grouped 两种候选扫描组织方式。该表不表示硬件峰值，只用于解释本实现中不同 kernel 的瓶颈差异。

阶段	有效工作量	对应耗时	派生指标	结论
手写 GEMM score	38.4 GFLOP	240.08 ms	160.0 GFLOP/s	16 × 16 tiled kernel 结构清晰，但距离 cuBLAS 仍有明显差距。
cuBLAS score	38.4 GFLOP	69.47 ms	552.7 GFLOP/s	说明矩阵乘法阶段可以通过成熟库获得直接收益。
线程 0 Top- k	2.0 亿个 score	226.56 ms	0.88 Gscore/s	顺序合并候选成为和 score 同量级的瓶颈。
树形 Top- k	2.0 亿个 score	91.92 ms	2.18 Gscore/s	共享内存树形合并降低了 block 末端串行部分。
IVF-4 per-query search	约 5000 万候选	922.68 ms	52.0 GFLOP/s	候选 list 随 query 分散，kernel 形态不规则。
IVF-4 grouped search	约 5000 万候选	64.04 ms	749.6 GFLOP/s	按簇分桶后能以小批量 GEMM 扫描同一 list，显著改善候选阶段。

表 26 由重复运行日志推导的 GPU 阶段吞吐与组织方式对比

上述结果表明，GEMM 和 IVF 的差异不仅在于“全量扫描”与“近似候选扫描”。GEMM 的 score 阶段访存和计算形态规则，可以通过共享内存 tile 或 cuBLAS 获得稳定复用；Top- k 的候选合并同样会成为全量扫描的主要瓶颈。IVF 的候选 list 则随 query 的 probe 结果变化，list 长度和访问位置都不完全一致；当按 list 分桶后，同一个簇内的向量矩阵可以被多个 query 共享扫描，虽然多了重排和合并开销，但能显著减少不规则 per-query 扫描带来的浪费。

7 实验分析

7.1 GEMM 路径分析

`gemm` 模式的主要计算量为 $O(NM_cD)$ 。在 DEEP100K 中, N 远大于 D , 因此 kernel 的并行度主要来自 base 行方向; batch query 又在另一个维度提供并行度。与单 query 相比, batch 方式可以把 base 常驻显存, 并让同一份 base tile 被多个 query 输出复用。

显存占用由四部分构成: base 常驻数组、query chunk、score tile 和 Top-10 输出。score tile 是最大临时数组, 因此 chunk 大小需要在并行度和显存占用之间折中。本实现默认 `query_chunk=128`, 此时 score tile 大小约为 $128 \times 100000 \times 4 \approx 48.8$ MiB, 既能提供足够 query 并行度, 又不会接近常见消费级 GPU 显存上限。实验结果表明, 手写 tiled GEMM 更适合作为 CUDA tile 设计的说明性实现; cuBLAS 更适合作为高性能基线。二者共用同一 Top- k 后处理, 有助于区分 score 计算和候选选择两个瓶颈。

7.2 Top- k 路径分析

Top- k kernel 的扫描复杂度为 $O(M_cN)$, 但其输出规模只有 $O(M_ck)$ 。由于 $k = 10$ 固定且较小, 线程局部数组可以放在寄存器中维护; block 内合并时共享内存占用为

$$\text{threads} \times k \times (\text{sizeof(float)} + \text{sizeof(uint32_t)}).$$

`ClampThreads` 会根据设备共享内存限制裁剪线程数, 避免动态共享内存超过单 block 限制。该设计比完整排序更符合 ANN Top- k 场景: 只保留前 10 个候选, 既减少计算, 也减少回传。基础版 Top- k 暴露出线程 0 顺序合并的瓶颈, 树形合并版则用更多线程参与 block 内候选归并, 使 Top- k 阶段耗时降低到基础版的约 $0.41\times$ 。

7.3 IVF 路径分析

IVF 模式的在线开销可分为 centroid scoring、probe selection、candidate search 和最终 Top- k 合并。centroid 数量 $nlist$ 通常远小于 base 数量, 因此 per-query 基线中第一阶段开销较小; 第二阶段开销取决于 `nprobe` 个 list 的总长度。若 `nprobe` 增大, 召回率通常上升, 但候选扫描时间也增加; 若 `nprobe` 过小, 搜索速度提升, 但容易漏掉真实近邻所在簇。

per-query 路径的优点是实现简单, 每个 query block 直接读取自己的 probe list; 缺点是不同 query 的候选集合不一致, GPU 上的访存和控制流更不规则。grouped-IVF 则把 batch 内 query 按 probe 簇编号重新分组, 使同一个 list 的候选向量能够写成矩阵并被一组 query 共同扫描。表 24 显示, grouped-IVF 在相同 `nprobe` 下几乎不改变 Recall, 却显著降低候选扫描时间。由此可见, 作业提示中的分组策略并非单纯的工程性重排, 而是能改变候选扫描 kernel 形态的关键设计。

7.4 数据传输与设备端计算

本实验把计时字段拆成 copy、score/probe、topk/search 和 result copy。该拆分用于判断瓶颈属于数据传输还是 kernel 计算。如果 query copy 和 result copy 占比较高, 通常表明 batch 太小或传输开销未被计算充分覆盖; 如果 score/probe compute 占比高, 则表明 GEMM、cuBLAS 或

grouped-IVF 的分组扫描成为主瓶颈；如果 `topk/search compute` 占比高，则应重点检查 Top- k 合并或 IVF 候选列表长度。表 23 中，基础 GEMM 的 `score` 和 Top- k 都不可忽略；表 24 中，per-query IVF 的主要设备端耗时在候选 list 扫描，而 grouped-IVF 把该部分显著压缩，但会增加分桶、query/result 拷贝和主机合并开销。

加速比结果还反映出 setup 口径的重要性。GEMM 的 online 加速比高于 wall 加速比，是因为 base 首次拷贝、cuBLAS handle、结果缓冲分配和主机端调度在单次运行中不可忽略；IVF 的差距更明显，因为构建 coarse list 和倒排表需要额外准备时间。若实际系统连续处理多个 query batch，base 或 IVF index 可以常驻显存，online 口径更能反映吞吐；若只运行一次命令程序，wall 口径更保守。

7.5 静态 profile 与汇编分析

本机 CUDA Toolkit 未安装 `nsys` 和 `ncu` 命令行工具，因此本节使用三类可复现证据进行性能剖析：第一，正文输出的 CUDA event 分段计时；第二，`cuobjdump --dump-resource-usage build/main.exe` 导出的 kernel 静态资源；第三，`nvcc --ptx` 与 `cuobjdump --dump-sass` 导出的 PTX/SASS 指令片段。云端环境虽可使用 Nsight Systems，但本次实验未在同一代码与数据路径下重新运行云端 timeline，因此本文未将 `nsys` 图作为定量分析依据。资源占用汇总见表 27。

Kernel	Reg	Stack/B	Static shared/B	Local/B	含义
ScoreGemmKernel	27	0	2048	0	两个 16×16 float tile，各 1024 B；静态 shared memory 与源码 tile 设计一致。
ScoreTopKKernel	32	80	0	0	基础 Top- k 保留线程局部数组，仍有少量栈用量。
ScoreTopKTreeKernel	32	0	0	0	树形合并版消除了静态栈用量，动态共享内存由 launch 时传入。
ScoreTopKWithIdsTreeKernel	32	0	0	0	grouped-IVF 使用的带全局 id Top- k ，资源形态与树形合并版接近。
IvfSearchKernel	31	160	0	0	per-query IVF 每线程维护局部 Top-10，同时扫描候选 list，栈用量高于 GEMM。
SelectProbeKernel	32	256	0	0	单线程选择 <code>nprobe</code> ，数组上限 32，开销小但并行度有限。

表 27 CUDA kernel 静态资源占用，来源于 `cuobjdump_resource.txt`

PTX 输出中，`ScoreGemmKernel` 明确包含两个静态共享内存数组：

```

1 .shared .align 4 .b8 base_tile[1024];
2 .shared .align 4 .b8 query_tile[1024];
3 ld.global.nc.f32 ... // base/query global load
4 st.shared.f32 ... // write tile into shared memory
5 bar.sync 0;
6 ld.shared.f32 ...
7 fma.rn.f32 ...
8 bar.sync 0;
```

该片段与源码中的 16×16 tiled GEMM 一致。每轮 tile 先从全局内存读入 base 和 query，再写入共享内存；`bar.sync` 对应 `__syncthreads()`，保证 tile 完整后再执行 16 次 FMA 累加。SASS 中同一位置可见 LDG、STS、LDS、FFMA 与 `BAR.SYNC` 指令，说明编译器将内层乘加转成 fused multiply-add，并使用共享内存作为片上复用缓冲。对应的 SASS 片段如下，STS 将全局加载后的数据写入共享内存，`BAR.SYNC` 对应 tile 装载后的块内同步，LDS 从共享内存读取 tile 数据，FFMA 执行乘加累积：

```

1 /*0310*/ STS [R22], R27;
2 /*0320*/ STS [R22+0x400], R29;
3 /*0330*/ BAR.SYNC.DEFER_BLOCKING 0x0;
4 /*0340*/ LDS R8, [R21];
5 /*03f0*/ FFMA R12, R12, R8, R9;
6 /*0580*/ BAR.SYNC.DEFER_BLOCKING 0x0;

```

`ScoreTopKKernel` 的 PTX/SASS 则呈现另一种形态：扫描阶段包含连续 `ld.global.nc.f32`，随后将每线程的 10 个候选写入动态共享内存。基础版末端由线程 0 读取共享内存并完成最终排序；树形版把候选合并拆成多轮共享内存归并，因此资源表中 `ScoreTopKTreeKernel` 的静态栈用量降为 0。结合图 5 和表 26，全量扫描不仅受矩阵乘法约束，Top-10 后处理也已经成为同量级开销。

IVF profile 结果更集中地反映候选组织方式的影响：per-query 路径中 `topk/search` 随 `nprobe` 增大快速上升，`nprobe=4` 时达到 922.68 ms；grouped-IVF 在相同 `nprobe=4` 下把该阶段降到 64.04 ms，但 `score/probe`、`query copy` 和 `result copy` 字段增加，反映出分桶、重排和主机端合并的额外代价。该现象表明，当前 IVF 在线阶段的主要矛盾不是 centroid 数量，而是候选 list 如何组织成适合 GPU 的规则批量计算。

8 总结

任务一 HIP 云学习系统梳理了 GPU 编程的共性方法：访存合并决定带宽型 kernel 的上限，共享内存适合做片上复用和重排，两级归约可以显著减少全局原子，warp/wave 内原语能够降低同步开销，多 kernel 编排则适合 K-Means 这类阶段性算法。

任务二 ANN GPU 实现完成了多条 GPU 搜索与对照路径。Flat 路径严格对应作业要求中的 batch 矩阵乘法，并进一步比较了手写 tiled GEMM、cuBLAS SGEMM、基础 Top- k 和树形 Top- k ；IVF 路径在 per-query probe list 基线之外，实现了按 probe 簇分桶的 grouped-IVF，对同一簇内向量矩阵与 query 小 batch 做 GEMM 式扫描。实测结果表明，`cublas_tree` 在 Recall@10 接近 1 时达到 83.24 $\mu\text{s}/\text{query}$ ；grouped-IVF 在 `nprobe=4` 时以 Recall@10 = 0.96280 换取 145.32 $\mu\text{s}/\text{query}$ ，在 `nprobe=8` 时以 189.90 $\mu\text{s}/\text{query}$ 达到 Recall@10 = 0.99345。CUDA event、重复测量、`cuobjdump` 资源占用与 PTX/SASS 片段共同表明：ANN GPU 优化除距离计算外，还受到 Top- k 合并和 IVF 候选组织方式的制约。

参考文献

- [1] NVIDIA Corporation. *CUDA C++ Programming Guide*. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>.
- [2] NVIDIA Corporation. *CUDA C++ Best Practices Guide*. <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/>.
- [3] AMD ROCm Documentation. *Introduction to the HIP Programming Model*. https://rocm.docs.amd.com/projects/HIP/en/latest/understand/programming_model.html.
- [4] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021. arXiv:1702.08734. <https://arxiv.org/abs/1702.08734>.
- [5] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1):117–128, 2011. doi:10.1109/TPAMI.2010.57.
- [6] Artem Babenko and Victor Lempitsky. Efficient indexing of billion-scale datasets of deep descriptors. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2055–2063, 2016. https://openaccess.thecvf.com/content_cvpr_2016/html/Babenko_Efficient_Indexing_of_CVPR_2016_paper.html.
- [7] 并行程序设计课程 GPU 编程作业要求、AMD AUP HIP Lab1–Lab6 notebook 与课程 ANN 实验框架。
- [8] 本仓库前序 ANN SIMD 实验结果目录 `ann-SIMD/bench_results/windows_i9_13900h/` 中的 `baseline_flat.csv` 与 `RESULTS.md`。

A 学习通过截图

本附录给出 AMD AUP 平台六个 Lab 的学习通过截图。部分原图为终端横向长截图，直接按整图缩放会导致文字偏小；因此，`report/fig/` 中放置关键输出区域的放大图，完整原始截图保存在 `ann-gpu/img/` 目录中。

```
jovyan@jupyter-github-kyc001---b91cab34:/opt/workspace/HIP/Lab1_VectorAddition:~$  
echo "5 1.5 2.3 3.7 4.1 5.9 0.8 1.2 2.3 3.4 4.6" | ./main  
2.300 3.500 6.000 7.500 10.500  
jovyan@jupyter-github-kyc001---b91cab34:/opt/workspace/HIP/Lab1_VectorAddition:~$
```

图 6 Lab1 Vector Addition 学习通过截图关键区域

```
jovyan@jupyter-github-kyc001---b91cab34:/opt/workspace/HIP/Lab2_MatrixTranspose:~$  
echo "2 3 1.0 2.0 3.0 4.0 5.0 6.0" | ./main  
1 4 2  
5 3 6  
jovyan@jupyter-github-kyc001---b91cab34:/opt/workspace/HIP/Lab2_MatrixTranspose:~$
```

图 7 Lab2 Matrix Transpose 学习通过截图关键区域

```
jovyan@jupyter-github-kyc001---b91cab34:/opt/workspace/HIP/Lab3_Histogram$
echo "8 4 0 1 2 3 0 1 2 0" | ./main
3 2 2 1
jovyan@jupyter-github-kyc001---b91cab34:/opt/workspace/HIP/Lab3_Histogram$
```

图 8 Lab3 Histogram 学习通过截图关键区域

```
jovyan@jupyter-github-kyc001---b91cab34:/opt/workspace/HIP/Lab4_Reduction$ cd /opt/workspace/HIP/Lab4_Reduction

# 1. 备份原文件，再修复启动参数（把 block 数和线程数交换回来）
cp main.hip main.hip.bak
sed -i 's/<<

```

图 9 Lab4 Reduction 学习通过截图

```
jovyan@jupyter-github-kyc001---b91cab34:/opt/workspace/HIP/Lab5_Monte_Carlo_Integration$ cd /
hipcc -O3 -Wno-unused-value main.hip -o main
echo "0 2 8 0.0625 0.25 0.5625 1.0 1.5625 2.25 3.0625 4.0" | ./main
# →3.1875
3.1875
jovyan@jupyter-github-kyc001---b91cab34:/opt/workspace/HIP/Lab5_Monte_Carlo_Integration$
```

图 10 Lab5 Monte Carlo 学习通过截图关键区域

```
jovyan@jupyter-github-kyc001---b91cab34:/opt/workspace/HIP/Lab6_K-Means_Clustering$ cd /opt/workspace/H
cp main.hip main.hip.bak
perl -0pi -e 's/(\n\s*hipFree\(\d_data_x\);)/\n      {\n          std::vector<int> outlabels(sample_size);\n          ::cout << outlabels[i] << " "; \n          std::cout << "\n"; \n      }$1/' main.hip
hipcc -O3 -Wno-unused-value main.hip -o main
echo "4 2 10 1.0 2.0 8.0 9.0 1.0 2.0 8.0 9.0 1.0 8.0 1.0 8.0" | ./main
# 期望三行: 1.5 8.5 / 1.5 8.5 / 0 0 1 1
1.5 8.5
1.5 8.5
0 0 1 1
jovyan@jupyter-github-kyc001---b91cab34:/opt/workspace/HIP/Lab6_K-Means_Clustering$ █
```

图 11 Lab6 K-Means 学习通过截图关键区域

B AI 使用报告

实验过程中，生成式 AI 仅用于作业要求梳理、实现路线比较、代码结构检查和文字表达调整。实验数据、运行结果和性能结论均来自课程材料、本地编译运行与程序输出；可复现入口以 `main.cc`、`Makefile` 和正文列出的命令为准。

使用场景	使用目的	采用情况
作业范围确认	明确 HIP 在线学习、ANN GPU 加速以及页数限制之间的关系。	报告划分为两个独立任务：HIP 填空和截图作为基础要求材料，ANN 主体按进阶选题研究报告组织。
ANN GPU 路线	比较单 query 与 batch query 的 GPU 实现方式，并评估完整 score 矩阵回传的必要性。	采用 batch GEMM 与设备端 Top-10 合并，减少主机端排序和完整得分矩阵回传。
工程结构与模块化	保持 <code>main.cc</code> 框架尽量不改，并采用前几次实验的 header-only 风格。	保留主程序薄入口，将 CUDA 公共工具、GEMM kernel、Top- <i>k</i> kernel、GEMM 搜索和 IVF 搜索拆分到 <code>gpu/</code> 目录，由 <code>gpu_ann_search.cuh</code> 统一聚合。
性能分析取舍	确定本次 ANN GPU 选题的性能分析范围。	保留 CUDA event 分段计时和 correctness 验证，正文围绕数据传输、score/probe kernel 和 Top- <i>k</i> /search kernel 展开分析。
本地工具链问题	定位 Windows 下 <code>nvcc</code> 编译失败的原因并完成本地验证。	确认失败原因是缺少 <code>cl.exe</code> host compiler；随后补齐 Visual Studio Build Tools 到 <code>C:\BuildTools</code> ，修改 <code>Makefile</code> 自动加载 <code>vcvars64.bat</code> ，并完成 <code>gemm 2000</code> 与 <code>ivf 2000 16 4</code> 的本地运行验证。

表 28 生成式 AI 辅助使用记录